



山东大学
SHANDONG UNIVERSITY

计算机系统原理实验报告

第四次实验

基于 RISC-V 开发板的加密算法性能测试与加速比分析

成员：	江李阳	202400460042
	王健一	202400460037
	邱泽辉	202400460046
	史淇	202333180074
专业：	密码科学与技术	
日期：	June 11, 2026	

Contents

1	实验目的	2
2	实验环境	2
2.1	开发板硬件环境	2
2.2	开发板软件环境	2
2.3	主机环境	3
3	实验原理	3
3.1	SM4 算法	3
3.1.1	加密算法	3
3.1.2	轮密钥生成方案	4
3.1.3	解密算法	4
4	实验过程	4
4.1	实验环境搭建	4
4.2	SM4 示例程序的测试使用	6
4.3	SM3 示例程序的使用测试	7
4.4	SM4 默认循环与循环展开	7
4.4.1	SM4 默认循环	7
4.4.2	SM4 循环展开	8
4.5	SM4 默认 XBox 查表与 XBox 合并	9
4.5.1	SM4 默认 XBox 查表	10
4.5.2	XBox 合并	11
4.6	SM4 默认轮函数连续异或与减少指令依赖	13
4.7	默认字节序转换与 RISC-V B 拓展 rev8 转换	14
5	性能测试对比	17
6	实验结果分析	19
6.1	整体性能对比	19
6.2	查表优化的压倒性优势	19
6.3	微架构优化的有限效果	20
6.3.1	循环展开 (unrolled)	20
6.3.2	降低指令依赖 (reduced_dep)	20
6.4	ISA 扩展的独立与叠加效应	20
6.5	数据规模对加速比的影响	21
6.6	小结与建议	21

1 实验目的

1. 学习并掌握 RISC-V 加架构的开发板 (香橙派) 的基本使用方法。
2. 学习 OpenHiTLS 加密算法的使用示例, 能正确运行 SM3 或 SM4 实例。
3. 记录各方案的运行时间, 吞吐率以及加速比。
4. 结合技术方案差异、开发板硬件条件、系统环境和测试方法等, 对实验结果进行分析。
5. 若某一技术方案依赖开发板不支持的指令扩展, 应说明原因并选择替代方案。

2 实验环境

2.1 开发板硬件环境

- 开发板型号: Orange Pi RV2
- 处理器: Ky(R) X1 (RISC-V 64 位)
 - 架构: riscv64 (Little Endian)
 - ISA 扩展: rv64imafdcv_zicsr_zifencei_zihintpause_zba_zbb_zbc_zbs_zicbop_zicboz
 - 处理器核心: 8 核 (1 Socket $\times$ 8 Cores)
 - 每核心线程数: 1
 - CPU 频率范围: 614.4 MHz (最低) — 1600.0 MHz (最高)
- 缓存:
 - L1d Cache: 256 KiB (8 实例, 即每核心 32 KiB)
 - L1i Cache: 256 KiB (8 实例, 即每核心 32 KiB)
 - L2 Cache: 1 MiB (2 实例, 即每 4 核心共享 512 KiB)
- 内存: 7.7 GiB (无 Swap)

2.2 开发板软件环境

- 操作系统: Ubuntu 24.04
- 交叉编译器: riscv64-linux-gnu-gcc / g++ 13.3.0
- 构建工具: CMake 3.28.3、GNU Make 4.3
- OpenHiTLS: (待补充版本/commit)
- libboundscheck: (待补充版本)

2.3 主机环境

- 主机硬件
 - 处理器:13th Gen Intel(R) Core(TM) i5-13420H
 - 内存:16 GB
- 主机软件
 - CP210x Windows Drivers(USB 转 TTL 串口芯片驱动)
 - MobaXterm(终端管理工具)
 - OpenSSH_for_Windows_9.5p2, LibreSSL 3.8.2(远程访问开发板)

3 实验原理

3.1 SM4 算法

SM4 是我国商用密码标准 (GB/T 32907-2016) 中规定的分组密码算法, 采用非平衡 Feistel 结构, 分组长度为 128 bit, 密钥长度为 128 bit, 迭代轮数为 32 轮。

3.1.1 加密算法

输入 128bit 明文 m , 按 32-bit 分为 4 个字 $X_0^0, X_0^1, X_0^2, X_0^3$ 进行 32 轮迭代, 每轮迭代都有一个 32bit 的轮密钥 K_{i-1} 参与计算, 第 $i (i = 1, 2, \dots, 32)$ 轮的计算如下:

$$\begin{aligned} X_i^0 &= X_{i-1}^1 \\ X_i^1 &= X_{i-1}^2 \\ X_i^2 &= X_{i-1}^3 \\ X_i^3 &= X_{i-1}^0 \oplus T(X_{i-1}^1 \oplus X_{i-1}^2 \oplus X_{i-1}^3 \oplus K_{i-1}) \end{aligned}$$

对最后一轮迭代的输出进行反序变化 R 得到 128bit 密文 c :

$$c = (Y^0, Y^1, Y^2, Y^3) = R(X_{32}^0, X_{32}^1, X_{32}^2, X_{32}^3) = (X_{32}^3, X_{32}^2, X_{32}^1, X_{32}^0)$$

其中函数 $T: \mathbb{F}_2^{32} \rightarrow \mathbb{F}_2^{32}$ 由非线性变换 S 和线性变换 L 组成:

$$T = L \circ S$$

设函数 T 的 32bit 输入为 $A = (a_0, a_1, a_2, a_3) \in (\mathbb{F}_{2^8})^4$, 则

- 非线性变换 S : 对每个字节单独进行查表代替操作, 即

$$B = (S(a_0), S(a_1), S(a_2), S(a_3))$$

类似 AES 加密, SM4 算法采用 1 个 S 盒, 查表时将每个字节的高 4bit 看作行标, 低 4bit 看作列标, 对应取值即为输出字节。

- 线性变换 L : 对非线性变换 S 的 32bit 输出 B , 计算

$$L(B) = B \oplus (B \lll 2) \oplus (B \lll 10) \oplus (B \lll 18) \oplus (B \lll 24)$$

其中 $\lll x$ 表示循环左移 x bit。

3.1.2 轮密钥生成方案

SM4 算法的主密钥 K 为 128bit, 轮密钥生成方案也采用非平衡 Feistel 结构。轮密钥 K_i 由主密钥 K 通过密钥扩展算法生成, 具体做法为: 先将主密钥 K 按 32-bit 分为 4 个字 MK_0, MK_1, MK_2, MK_3 , 然后与 32-bit 常量 FK_i 进行异或得到初始轮密钥:

$$K_{-4} = MK_0 \oplus FK_0$$

$$K_{-3} = MK_1 \oplus FK_1$$

$$K_{-2} = MK_2 \oplus FK_2$$

$$K_{-1} = MK_3 \oplus FK_3$$

其中 $FK_0 = 0xA3B1BAC6, FK_1 = 0x56AA3350, FK_2 = 0x677D9197, FK_3 = 0xB27022DC$ 。接着通过迭代计算得到后续轮密钥:

$$K_i = K_{i-4} \oplus T'(K_{i-3} \oplus K_{i-2} \oplus K_{i-1} \oplus CK_i), \quad i = 0, 1, \dots, 31$$

其中 CK_i 为固定参数; T' 函数与 T 函数类似, 但线性变换部分不同, 仅需替换 L 为 L' :

$$T'(X) = L'(S(X))$$

$$L'(B) = B \oplus (B \lll 13) \oplus (B \lll 23)$$

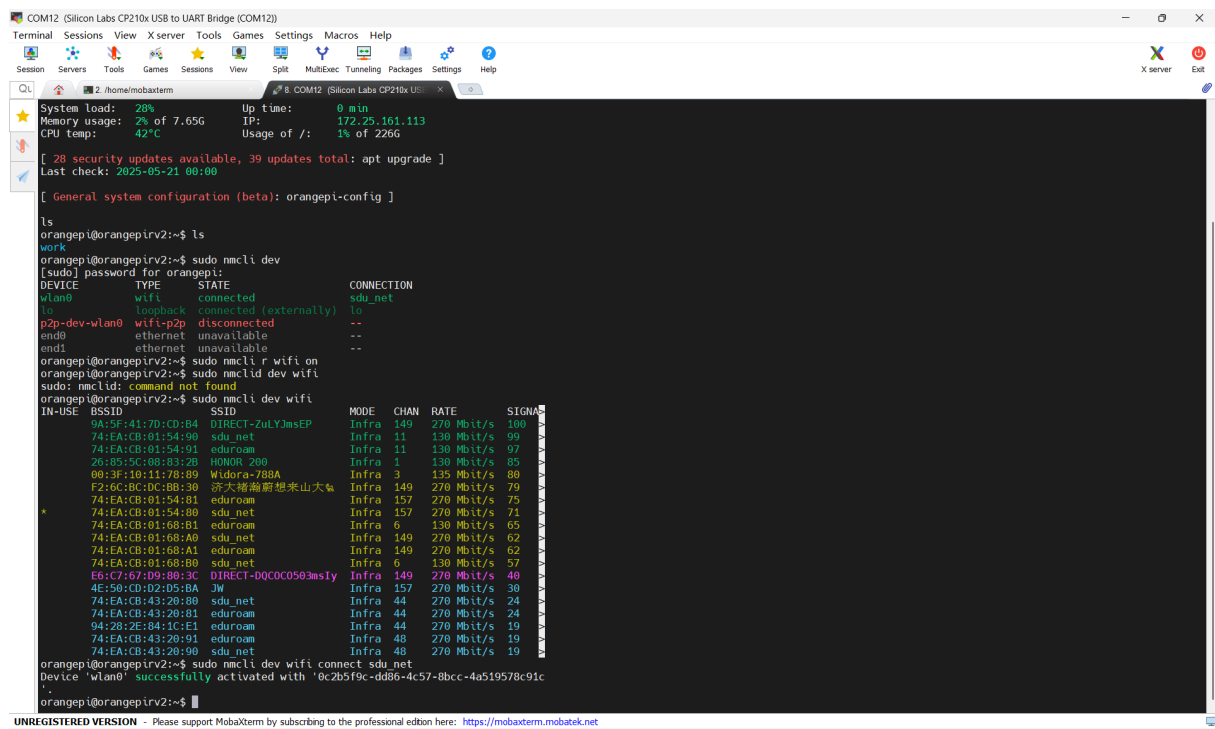
3.1.3 解密算法

因为采用广义 Feistel 结构, 所以 SM4 算法具有加解密结构一致性, 因此解密算法与加密算法一致, 仅需将参与第 i ($i = 1, 2, \dots, 32$) 轮迭代运算的轮密钥由 K_{i-1} 替换为 K_{32-i} 即可。

4 实验过程

4.1 实验环境搭建

为香橙派开发板插上 TYPEC 电源适配器并用 USB 转 TTL 模块连接开发板与主机, 连接完成后使用 MobaXterm 访问开发板, 终端连接 WIFI 成功如图:



```
COM12 (Silicon Labs CP210x USB to UART Bridge (COM12))
Terminal Sessions View Xserver Tools Games Settings Macros Help
Session Servers Tools Games Sessions View Split MultiExec Tunneling Packages Settings Help

System load: 20% Up time: 0 min
Memory usage: 2% of 7.65G IP: 172.25.161.113
CPU temp: 42°C Usage of /: 1% of 228G

[ 28 security updates available, 39 updates total: apt upgrade ]
Last check: 2025-05-21 00:00

[ General system configuration (beta): orangepi-config ]

ls
orangepi@orangepi2:~$ ls
work
orangepi@orangepi2:~$ sudo nmcli dev
[sudo] password for orangepi:
DEVICE TYPE STATE CONNECTION
wlan0 wifi connected (externally) sdu_net
lo loopback connected (externally) lo
p2p-dev-wlan0 wifi-p2p disconnected --
end0 ethernet unavailable --
orangepi@orangepi2:~$ sudo nmcli r wifi on
orangepi@orangepi2:~$ sudo nmcli dev wifi
sudo: nmcli: command not found
orangepi@orangepi2:~$ sudo nmcli dev wifi
IN-USE BSSID SSID MODE CHAN RATE SIGNAL
74:EA:CB:01:54:90 sdu_net Infra 11 130 Mbit/s 99
74:EA:CB:01:54:91 eduroam Infra 11 130 Mbit/s 97
20:85:5C:08:83:2B HONOR 200 Infra 1 130 Mbit/s 85
08:3F:10:11:70:09 Wldora-700A Infra 3 135 Mbit/s 88
F2:6C:8C:DC:88:30 科大洛瀚斯想来山大 WiFi Infra 149 270 Mbit/s 78
74:EA:CB:01:54:81 eduroam Infra 157 270 Mbit/s 75
74:EA:CB:01:54:80 sdu_net Infra 157 270 Mbit/s 71
74:EA:CB:01:68:B1 eduroam Infra 6 130 Mbit/s 65
74:EA:CB:01:68:A0 sdu_net Infra 149 270 Mbit/s 62
74:EA:CB:01:68:A1 eduroam Infra 140 270 Mbit/s 62
74:EA:CB:01:68:B0 sdu_net Infra 6 130 Mbit/s 57
E6:C7:67:D9:80:3C DIRECT-DQCO0503msly Infra 149 270 Mbit/s 48
4E:58:CD:D2:D5:BA JW Infra 157 270 Mbit/s 38
74:EA:CB:43:20:80 sdu_net Infra 44 270 Mbit/s 24
74:EA:CB:43:20:81 eduroam Infra 44 270 Mbit/s 24
94:2B:2E:84:1C:E1 eduroam Infra 44 270 Mbit/s 19
74:EA:CB:43:20:91 eduroam Infra 48 270 Mbit/s 19
74:EA:CB:43:20:90 sdu_net Infra 48 270 Mbit/s 19
orangepi@orangepi2:~$ sudo nmcli dev wifi connect sdu_net
Device 'wlan0' successfully activated with '0c2b5f9c-dd86-4c57-8bcc-4a519578c91c'.
orangepi@orangepi2:~$
```

Figure 1: 开发板连接 WIFI

考虑到原实验为串口连接开发板进行单人次实验, 而实际实验小组有多人, 轮流使用串口会显著增加实验总耗时, 不方便调试, 也不便于课后小组成员随时随地使用开发板的需要, 因此我们为开发板配置网络后启动了 OpenSSH 服务。基本实现为借助 Tailscale 工具将包括开发板在内的所有主机添加到同一个虚拟局域网中, 使得所有设备都可以跨局域网登录访问开发板:

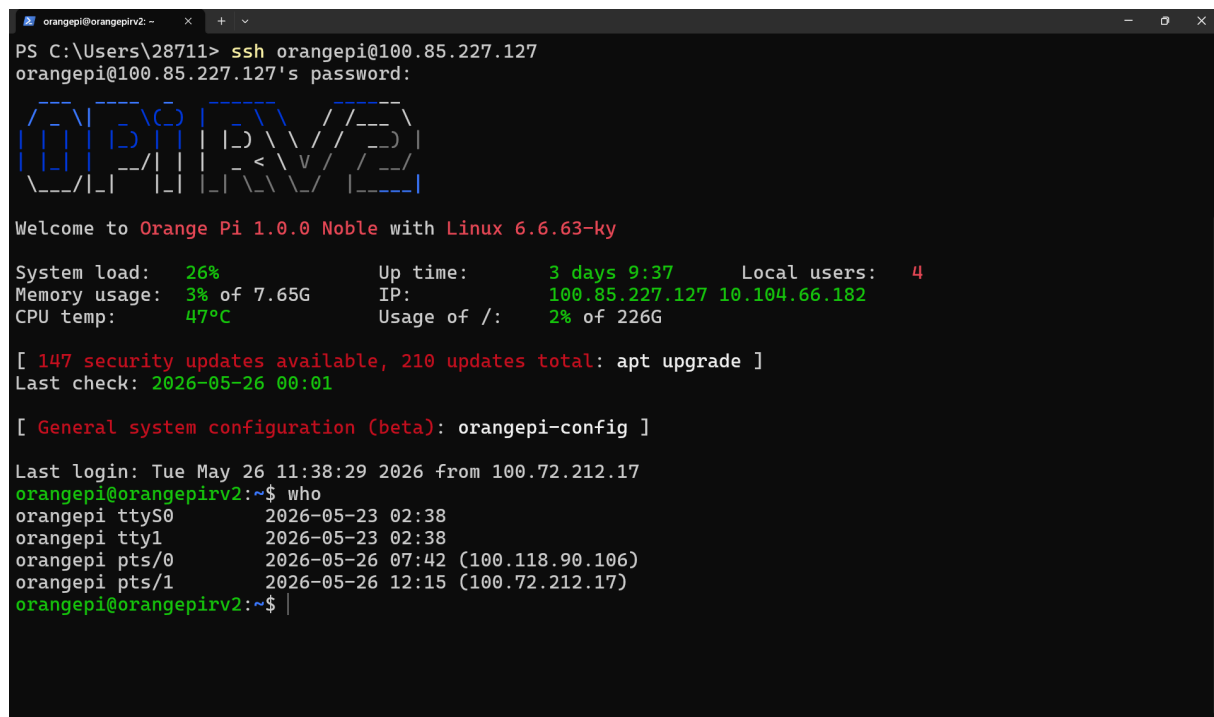


Figure 2: 远程连接开发板

这里远程连接只是为了方便多用户远程访问开发板和编写代码, 为确保实验结果准确性, 每次测试算法效率时仅一位用户在使用。

4.2 SM4 示例程序的测试使用

按照实验指导书的指示分别安装依赖、获取 libboundscheck 源码并编译、获取 openhitts 源码并编译、然后运行实例代码,结果如下图:

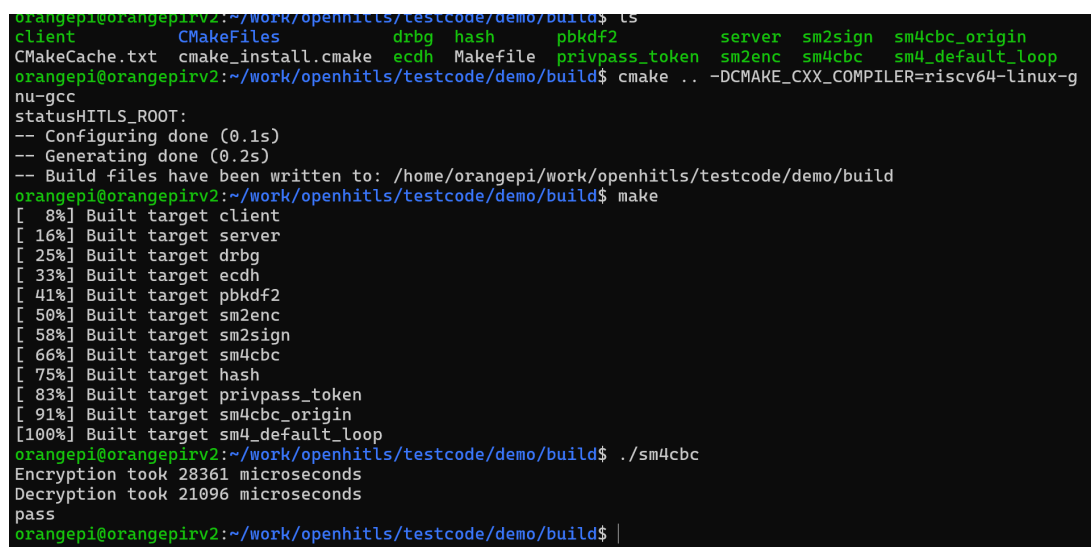


Figure 3: SM4 示例程序测试运行

4.3 SM3 示例程序的使用测试

修改 hash.c 文件中的部分代码, 重新编译即可测试运行 SM3 示例程序, 结果如下:

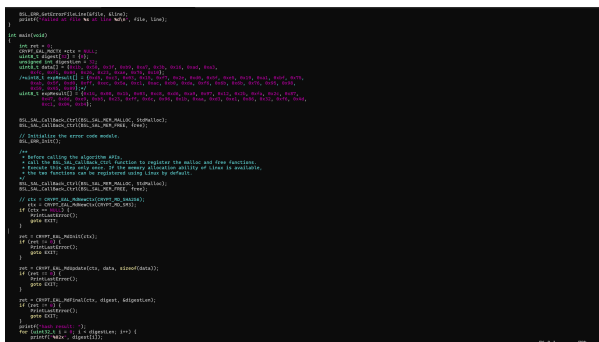
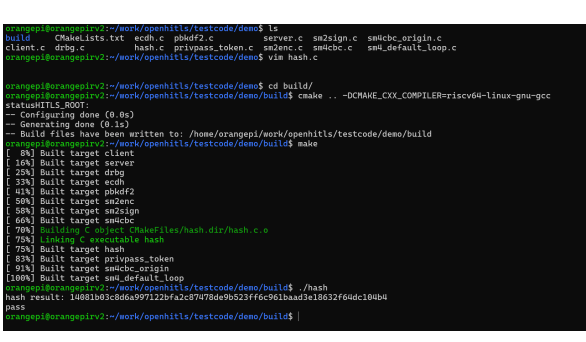



Figure 4: 修改代码

Figure 5: sm3 示例程序测试运行

4.4 SM4 默认循环与循环展开

4.4.1 SM4 默认循环

按照课本介绍 SM4 算法的实现, 不采用任何优化策略, 完整实现 SM4 的 32 轮迭代并在每一轮计算中对明文分组做非线性变换 S 和线性变换 L, 记录算法运行时间, 并以此组作为基准组, 时间开销主要在 32 轮迭代以及每轮迭代中的 S 和 L 操作:

openhits/testcode/demo/sm4_default_loop.c

```
1 static void sm4EncryptBlock(const uint32_t rk[32],
2                             const uint8_t in[16],
3                             uint8_t out[16])
4 {
5     uint32_t X[4];
6     sm4LoadBlock(in, X);
7
8     for (int i = 0; i < 32; i++) {
9         uint32_t tmp = X[1] ^ X[2] ^ X[3] ^ rk[i];
10        uint32_t t = sm4T(tmp);
11        uint32_t newX0 = X[0] ^ t;
12        X[0] = X[1];
13        X[1] = X[2];
14        X[2] = X[3];
15        X[3] = newX0;
16    }
17
18    sm4StoreBlock(X, out);
19 }
```

结果如下:

```

orangepi@orangepirv2:~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo/build$ ./sm4_default_loop
[PASS] sm4_default_loop: test vector correct
SM4 Default Loop - 1048576 bytes x 10 repeats
Elapsed : 445129 µs
Throughput: 22.465 MB/s
orangepi@orangepirv2:~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo/build$ |

```

其中测试输入数据大小为 1MB。

4.4.2 SM4 循环展开

上述的循环次数以及每次循环的操作是已知的, 使用宏一次性生成全部 32 轮代码, 可以去掉 32 次 `i++` 和条件判断, 避免分支预测失败惩罚, 同时再使用数组而是四个变量即可实现 32 轮循环迭代, 省去计算数组索引和基于偏移量取数的开销, 关键代码如下:

openhitls/testcode/demo/sm4_unrolled.c

```

1 static void sm4EncryptBlock(const uint32_t rk[32],
2                             const uint8_t in[16],
3                             uint8_t out[16])
4 {
5     uint32_t X0, X1, X2, X3;
6     uint32_t tmp, t, newX;
7
8     /* Manual load to scalars for unrolled access */
9     {
10         uint32_t arr[4];
11         sm4LoadBlock(in, arr);
12         X0 = arr[0]; X1 = arr[1]; X2 = arr[2]; X3 = arr[3];
13     }
14
15 #define SM4_UNROLLED_ROUND(N) do { \
16     tmp = X1 ^ X2 ^ X3 ^ rk[N]; \
17     t = sm4T(tmp); \
18     newX = X0 ^ t; \
19     X0 = X1; X1 = X2; X2 = X3; X3 = newX; \
20 } while(0)
21
22 SM4_UNROLLED_ROUND(0);
23 SM4_UNROLLED_ROUND(1);
24 SM4_UNROLLED_ROUND(2);
25 SM4_UNROLLED_ROUND(3);
26 SM4_UNROLLED_ROUND(4);
27 SM4_UNROLLED_ROUND(5);
28 SM4_UNROLLED_ROUND(6);
29 SM4_UNROLLED_ROUND(7);
30 SM4_UNROLLED_ROUND(8);
31 SM4_UNROLLED_ROUND(9);
32 SM4_UNROLLED_ROUND(10);
33 SM4_UNROLLED_ROUND(11);
34 SM4_UNROLLED_ROUND(12);
35 SM4_UNROLLED_ROUND(13);

```

```

36     SM4_UNROLLED_ROUND(14);
37     SM4_UNROLLED_ROUND(15);
38     SM4_UNROLLED_ROUND(16);
39     SM4_UNROLLED_ROUND(17);
40     SM4_UNROLLED_ROUND(18);
41     SM4_UNROLLED_ROUND(19);
42     SM4_UNROLLED_ROUND(20);
43     SM4_UNROLLED_ROUND(21);
44     SM4_UNROLLED_ROUND(22);
45     SM4_UNROLLED_ROUND(23);
46     SM4_UNROLLED_ROUND(24);
47     SM4_UNROLLED_ROUND(25);
48     SM4_UNROLLED_ROUND(26);
49     SM4_UNROLLED_ROUND(27);
50     SM4_UNROLLED_ROUND(28);
51     SM4_UNROLLED_ROUND(29);
52     SM4_UNROLLED_ROUND(30);
53     SM4_UNROLLED_ROUND(31);
54
55 #undef SM4_UNROLLED_ROUND
56
57     /* Reverse transform & store */
58     {
59         uint32_t arr[4] = {X0, X1, X2, X3};
60         sm4StoreBlock(arr, out);
61     }
62 }

```

这里所有宏会在编译阶段就计算出来, 能降低运行时计算的时间开销。运行结果如下:

```

orange@orangeiprv2: ~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo/build$ ls
sm4_default_loop  sm4_reduced_dep  sm4_unrolled  sm4_xbox  sm4_xbox_merged  sm4_xbox_zbb  sm4_zbb
orange@orangeiprv2:~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo/build$ ./sm4_unrolled
[PASS] sm4_unrolled: test vector correct
SM4 Loop Unrolled - 1048576 bytes x 10 repeats
Elapsed : 405184 μs
Throughput: 24.680 MB/s
orange@orangeiprv2:~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo/build$

```

Figure 6: SM4 循环展开

可以看到循环展开能有效提升 SM4 加密算法的运行效率, 提升吞吐量。

4.5 SM4 默认 XBox 查表与 XBox 合并

在 SM4 加密算法的每轮循环中需要对输入的每个字做非线性的过 S 盒操作, 和线性的 L 操作, 对于

$$X_3 = a_0 || a_1 || a_2 || a_3$$

为最后一个 32bit 分组, 分别过 S 盒后得到

$$B = b_0 || b_1 || b_2 || b_3 = Sbox(a_0) || Sbox(a_1) || Sbox(a_2) || Sbox(a_3)$$

在代码中通过移位拼接:

$$B = (b_0 \ll 24) \oplus (b_1 \ll 16) \oplus (b_2 \ll 8) \oplus (b_3)$$

由于 L 是线性操作, 满足

$$L(a \oplus b) = L(a) \oplus L(b)$$

因此

$$\begin{aligned} L(B) &= L(b_0 \ll 24) \oplus L(b_1 \ll 16) \oplus L(b_2 \ll 8) \oplus L(b_3) \\ &= B \oplus \text{ROL}(B, 2) \oplus \text{ROL}(B, 10) \oplus \text{ROL}(B, 18) \oplus \text{ROL}(B, 24) \\ &= \text{ROL}(\text{Xbox}[b_0], 24) \oplus \text{ROL}(\text{Xbox}[b_1], 16) \oplus \text{ROL}(\text{Xbox}[b_2], 8) \oplus \text{Xbox}[b_3] \end{aligned} \quad (1)$$

4.5.1 SM4 默认 Xbox 查表

因此预计算 $\text{Xbox}[i] = L(S(i))$ 共 $2^8 = 256$ 项, 最后循环移位并异或即可得到最终结果。关键代码如下:

openhits/testcode/demo/sm4_xbox.c

```
1 static void sm4EncryptBlock(const uint32_t rk[32],
2                             const uint8_t in[16],
3                             uint8_t out[16])
4 {
5     uint32_t X0, X1, X2, X3;
6     {
7         uint32_t arr[4];
8         sm4LoadBlock(in, arr);
9         X0 = arr[0]; X1 = arr[1]; X2 = arr[2]; X3 = arr[3];
10    }
11
12    initXbox();
13
14    for (int i = 0; i < 32; i++) {
15        uint32_t t = X1 ^ X2 ^ X3 ^ rk[i];
16        uint8_t b0 = (uint8_t)((t >> 24) & 0xFF);
17        uint8_t b1 = (uint8_t)((t >> 16) & 0xFF);
18        uint8_t b2 = (uint8_t)((t >> 8) & 0xFF);
19        uint8_t b3 = (uint8_t)(t & 0xFF);
20
21        uint32_t newX0 = X0 ^ ROL(xbox[b0], 24)
22                        ^ ROL(xbox[b1], 16)
23                        ^ ROL(xbox[b2], 8)
24                        ^ xbox[b3];
25
26        X0 = X1; X1 = X2; X2 = X3; X3 = newX0;
27    }
28
29    {
30        uint32_t arr[4] = {X0, X1, X2, X3};
```

```

31         sm4StoreBlock(arr, out);
32     }
33 }

```

编译运行结果截图如下:

```

orange@orangeiprv2: ~/Prin
orangepi@orangeiprv2: ~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo/build$ ls
sm4_default_loop sm4_reduced_dep sm4_unrolled sm4_xbox sm4_xbox_merged sm4_xbox_zbb sm4_zbb
orangepi@orangeiprv2: ~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo/build$ ./sm4_xbox
[PASS] sm4_xbox: test vector correct
SM4 Xbox Table Lookup - 1048576 bytes x 10 repeats
Elapsed : 365158 μs
Throughput: 27.385 MB/s

```

Figure 7: SM4 默认 Xbox

效率较 SM4 默认循环和循环展开都有所提升, 不过这种提前计算会增大程序的空间复杂度。

4.5.2 Xbox 合并

观察 (1) 式可以发现最后循环移位的位数是固定的 (分别为 24、16、8、0), 可以分别预计算这里的固定位数循环移位, 将原本运行时计算 ROL 改为编译时计算且运行时查表, 实现 Xbox 合并。即计算:

$$\begin{cases} Xbox0[i] = L(S(i)) \\ Xbox1[i] = ROL(L(S(i)), 8) \\ Xbox2[i] = ROL(L(S(i)), 16) \\ Xbox3[i] = ROL(L(S(i)), 24) \end{cases}$$

那么实际运行时各个位置查对应的表即可:

$$\begin{cases} ROL(Xbox[b_0], 24) = Xbox3[b_0] \\ ROL(Xbox[b_1], 16) = Xbox2[b_1] \\ ROL(Xbox[b_2], 8) = Xbox1[b_2] \end{cases}$$

关键代码如下:

```

openhitls/testcode/demo/sm4_xbox_merged.c

1 static uint32_t XB0X0[256];
2 static uint32_t XB0X1[256];
3 static uint32_t XB0X2[256];
4 static uint32_t XB0X3[256];
5 static int xbox_merged_initialized = 0;
6
7 static inline uint32_t ROL(uint32_t x, int n)
8 {
9     return (x << n) | (x >> (32 - n));
10 }

```

```
11
12 static void initXboxMerged(void)
13 {
14     if (xbox_merged_initialized) return;
15
16     for (int i = 0; i < 256; i++) {
17         XBOX0[i] = sm4L((uint32_t)sm4Sbox((uint8_t)i));
18         XBOX1[i] = ROL(XBOX0[i], 8);
19         XBOX2[i] = ROL(XBOX0[i], 16);
20         XBOX3[i] = ROL(XBOX0[i], 24);
21     }
22     xbox_merged_initialized = 1;
23 }
24
25 static void sm4EncryptBlock(const uint32_t rk[32],
26                             const uint8_t in[16],
27                             uint8_t out[16])
28 {
29     uint32_t X0, X1, X2, X3;
30     {
31         uint32_t arr[4];
32         sm4LoadBlock(in, arr);
33         X0 = arr[0]; X1 = arr[1]; X2 = arr[2]; X3 = arr[3];
34     }
35
36     initXboxMerged();
37
38     for (int i = 0; i < 32; i++) {
39         uint32_t t = X1 ^ X2 ^ X3 ^ rk[i];
40         uint32_t newX0 = X0 ^ XBOX3[(t >> 24) & 0xFF]
41                         ^ XBOX2[(t >> 16) & 0xFF]
42                         ^ XBOX1[(t >> 8) & 0xFF]
43                         ^ XBOX0[t & 0xFF];
44
45         X0 = X1; X1 = X2; X2 = X3; X3 = newX0;
46     }
47
48     {
49         uint32_t arr[4] = {X0, X1, X2, X3};
50         sm4StoreBlock(arr, out);
51     }
52 }
```

编译运行后结果如下:

```

orangeipi@orangeiprv2:~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo/build$ ls
sm4_default_loop sm4_reduced_dep sm4_unrolled sm4_xbox sm4_xbox_merged sm4_xbox_zbb sm4_zbb
orangeipi@orangeiprv2:~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo/build$ ./sm4_xbo
x_merged
[PASS] sm4_xbox_merged: test vector correct
SM4 XBox Merged (4 tables) - 1048576 bytes x 10 repeats
Elapsed : 284581 μs
Throughput: 35.139 MB/s
orangeipi@orangeiprv2:~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo/build$ |

```

Figure 8: SM4XBox 合并

可以看到程序运行效率和吞吐量相较之前版本优化得到很大提升,但因为需要预计算和存储 4 张表,因此有总计 $4 * 256 * 4B = 4KB$ 的额外空间开销。

4.6 SM4 默认轮函数连续异或与减少指令依赖

在默认循环实现版本中每轮循环都有 $tmp = X1 \oplus X2 \oplus X3 \oplus rk[i]$, 由于异或操作满足结合律:

$$a \oplus b \oplus c \oplus d = (a \oplus b) \oplus (c \oplus d)$$

因此这里可以分别计算前两个变量的异或结果和后两个变量的异或结果, 分别存到临时变量中, 最后将这两个临时变量再求异或, 这样能减少指令之间的依赖性, 即可以同时求两个临时变量。否则, 连续异或时想要求后一个异或必须要等前一个异或运算结束, 指令依赖关系导致只能串行执行。在 SM4 每轮最后的换位赋值时可能产生写寄存器又马上读的依赖, 因此可以用临时变量存放 X_1, X_2 值, 这样能确保整个过程只发生寄存器移动而无需内存访问。具体代码如下:

openhitls/testcode/demo/sm4_reduced_dep.c

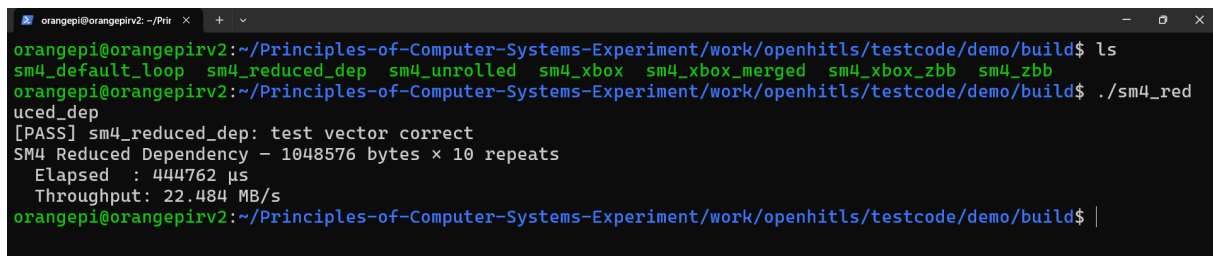
```

1 static void sm4EncryptBlock(const uint32_t rk[32],
2                             const uint8_t in[16],
3                             uint8_t out[16])
4 {
5     uint32_t X0, X1, X2, X3;
6     {
7         uint32_t arr[4];
8         sm4LoadBlock(in, arr);
9         X0 = arr[0]; X1 = arr[1]; X2 = arr[2]; X3 = arr[3];
10    }
11
12    for (int i = 0; i < 32; i++) {
13        /* Break the XOR chain: X1^X2 and X3^rk are independent */
14        uint32_t tmp1 = X1 ^ X2;
15        uint32_t tmp2 = X3 ^ rk[i];
16        uint32_t tmp = tmp1 ^ tmp2;
17        uint32_t t = sm4T(tmp);
18        uint32_t newX0 = X0 ^ t;
19
20        /* Feistel rotation with temporaries */
21        uint32_t prevX1 = X1;

```

```
22     uint32_t prevX2 = X2;
23     X0 = prevX1;
24     X1 = prevX2;
25     X2 = X3;
26     X3 = newX0;
27 }
28
29 {
30     uint32_t arr[4] = {X0, X1, X2, X3};
31     sm4StoreBlock(arr, out);
32 }
33 }
```

编译运行结果如图:



```
orangeipi@orangeiprv2: ~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo/build$ ls
sm4_default_loop sm4_reduced_dep sm4_unrolled sm4_xbox sm4_xbox_merged sm4_xbox_zbb sm4_zbb
orangeipi@orangeiprv2:~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo/build$ ./sm4_reduced_dep
[PASS] sm4_reduced_dep: test vector correct
SM4 Reduced Dependency - 1048576 bytes x 10 repeats
Elapsed   : 444762 µs
Throughput: 22.484 MB/s
orangeipi@orangeiprv2:~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo/build$ |
```

Figure 9: SM4 减少指令依赖

可以看到程序运行效率和吞吐量较默认循环版几乎相同, 优化非常有限。

4.7 默认字节序转换与 RISC-V B 拓展 rev8 转换

查看开发板 CPU 所支持的指令集如下图:

```

orangeipi@orangeiprv2:~/Principles-of-Computer-Systems-Experiment/work/openhits/testcode/demo/build$ cat /proc/cpuinfo
processor       : 0
hart           : 0
model name     : Ky(R) X1
isa            : rv64imafdcv_zicbom_zicboz_zicntr_zicnd_zicsr_zifencei_zihintpause_zihpm_zfh_zfhmin_zca_zcd_zba_zbb_zbc_zbs_zkt_zve32f_zve32x_
zve64d_zve64f_zve64x_zvfh_zvfhmin_zvkt_zscofpmf_zstc_zvinval_zvnapot_zvpbmt
mmu            : sv39
uarch          : ky, x60
mvendorid     : 0x710
marchid       : 0x8000000058000001
mimpid        : 0x1000000049772200

processor       : 1
hart           : 1
model name     : Ky(R) X1
isa            : rv64imafdcv_zicbom_zicboz_zicntr_zicnd_zicsr_zifencei_zihintpause_zihpm_zfh_zfhmin_zca_zcd_zba_zbb_zbc_zbs_zkt_zve32f_zve32x_
zve64d_zve64f_zve64x_zvfh_zvfhmin_zvkt_zscofpmf_zstc_zvinval_zvnapot_zvpbmt
mmu            : sv39
uarch          : ky, x60
mvendorid     : 0x710
marchid       : 0x8000000058000001
mimpid        : 0x1000000049772200

processor       : 2
hart           : 2
model name     : Ky(R) X1
isa            : rv64imafdcv_zicbom_zicboz_zicntr_zicnd_zicsr_zifencei_zihintpause_zihpm_zfh_zfhmin_zca_zcd_zba_zbb_zbc_zbs_zkt_zve32f_zve32x_
zve64d_zve64f_zve64x_zvfh_zvfhmin_zvkt_zscofpmf_zstc_zvinval_zvnapot_zvpbmt
mmu            : sv39
uarch          : ky, x60
mvendorid     : 0x710
marchid       : 0x8000000058000001
mimpid        : 0x1000000049772200

processor       : 3
hart           : 3
model name     : Ky(R) X1
isa            : rv64imafdcv_zicbom_zicboz_zicntr_zicnd_zicsr_zifencei_zihintpause_zihpm_zfh_zfhmin_zca_zcd_zba_zbb_zbc_zbs_zkt_zve32f_zve32x_
zve64d_zve64f_zve64x_zvfh_zvfhmin_zvkt_zscofpmf_zstc_zvinval_zvnapot_zvpbmt
mmu            : sv39
uarch          : ky, x60

```

Figure 10: 开发板 CPU 信息

可以看到该板支持 zbb, 因此可以在编译时启用该拓展。该拓展的优化原理在于:

- 对字节序转换的优化: 标准 SM4 中对串 01234567 的解释为:

01 23 45 67

即大端序, 而在 RISC-V 中却是小端序, 因此 sm4LoadBlock() 函数需要转换。普通实现时的代码

```

1 x =
2 (in[0]<<24)
3 |
4 (in[1]<<16)
5 |
6 (in[2]<<8)
7 |
8 in[3];

```

对应的汇编代码为:

```

1 lbu
2 slli
3
4 lbu
5 slli
6
7 lbu
8 slli

```

```

9
10 or
11 or
12 or

```

需要很多条指令才能完成, 而使用 Zbb 拓展中的 rev8 指令则可直接完成这一功能, 即直接

```

1 lw
2 rev8

```

即可完成, 使得如 sm4LoadBlock()、sm4StoreBlock() 函数得到加速。

- Zbb 对 L 变换的优化: 针 ROL(x, 10) 操作, 编译器默认生成:

```

1 slli t1,x,10
2 srli t2,x,22
3 or   t1,t1,t2

```

即:

$$(x \ll 10) | (x \gg 2)$$

这需要 2 次 shift 和 1 次 or 共 3 条指令, 对于 SM4 每轮的四次 ROL 则总共需要 12 条指令。Zbb 拓展提供 rori(Rotate Right Immediate) 指令, 可直接实现一次 ROL 操作, 因此每轮 L 变换仅需 4 条指令即可完成。

对比代码不同编译方式运行结果如下:

```

orangeip@orangeiprv2:~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo$ riscv64-linux-gnu-gcc \
-O2 \
-march=rv64gc \
-Wall -Wextra -std=c99 \
-D_POSIX_C_SOURCE=199309L \
-o sm4_zbb_nozbb \
sm4_zbb.c
orangeip@orangeiprv2:~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo$ ./sm4_zbb_nozbb
[PASS] sm4_zbb: test vector correct
SM4 Default Loop + Zbb - 1048576 bytes x 10 repeats
Elapsed : 444762 µs
Throughput: 22.484 MB/s
orangeip@orangeiprv2:~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo$ riscv64-linux-gnu-gcc \
-O2 \
-march=rv64gc_zba_zbb_zbc_zbs \
-Wall -Wextra -std=c99 \
-D_POSIX_C_SOURCE=199309L \
-o sm4_zbb_zbb \
sm4_zbb.c
orangeip@orangeiprv2:~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo$ ./sm4_zbb_zbb
[PASS] sm4_zbb: test vector correct
SM4 Default Loop + Zbb - 1048576 bytes x 10 repeats
Elapsed : 392394 µs
Throughput: 25.485 MB/s
orangeip@orangeiprv2:~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo$

```

Figure 11: SM4 默认字节序转换与 rev8 转换

可以看到拓展指令能小幅提升程序效率与吞吐量。

5 性能测试对比

针对上述对 SM4 加密算法性能的 7 种优化方案, 分别在 7 种不同输入数据规模 (1 KB 至 16 MB) 下测试程序的运行时间并计算吞吐率与加速比, 过程截图如下:

Data sizes: 1 KB, 16 KB, 64 KB, 256 KB, 1 MB, 4 MB, 16 MB			
--- 1 KB (1 KB, x2000 repeats) ---			
Running default_loop	... 88841 μ s		21.985 MB/s
Running unrolled	... 78583 μ s		24.854 MB/s
Running reduced_dep	... 88687 μ s		22.023 MB/s
Running xbox	... 68904 μ s		28.346 MB/s
Running xbox_merged	... 53699 μ s		36.372 MB/s
Running zbb_default	... 78315 μ s		24.939 MB/s
Running zbb_xbox	... 54234 μ s		36.013 MB/s
--- 16 KB (16 KB, x200 repeats) ---			
Running default_loop	... 142796 μ s		21.884 MB/s
Running unrolled	... 126458 μ s		24.712 MB/s
Running reduced_dep	... 142336 μ s		21.955 MB/s
Running xbox	... 110931 μ s		28.171 MB/s
Running xbox_merged	... 86501 μ s		36.127 MB/s
Running zbb_default	... 126095 μ s		24.783 MB/s
Running zbb_xbox	... 87517 μ s		35.707 MB/s
--- 64 KB (64 KB, x50 repeats) ---			
Running default_loop	... 143178 μ s		21.826 MB/s
Running unrolled	... 127542 μ s		24.502 MB/s
Running reduced_dep	... 142332 μ s		21.956 MB/s
Running xbox	... 111035 μ s		28.144 MB/s
Running xbox_merged	... 86475 μ s		36.138 MB/s
Running zbb_default	... 126130 μ s		24.776 MB/s
Running zbb_xbox	... 87639 μ s		35.658 MB/s
--- 256 KB (256 KB, x20 repeats) ---			
Running default_loop	... 228803 μ s		21.853 MB/s
Running unrolled	... 203336 μ s		24.590 MB/s
Running reduced_dep	... 228606 μ s		21.872 MB/s
Running xbox	... 178773 μ s		27.968 MB/s
Running xbox_merged	... 140014 μ s		35.711 MB/s
Running zbb_default	... 202658 μ s		24.672 MB/s
Running zbb_xbox	... 142426 μ s		35.106 MB/s
--- 1 MB (1 MB, x5 repeats) ---			
Running default_loop	... 229550 μ s		21.782 MB/s
Running unrolled	... 204633 μ s		24.434 MB/s
Running reduced_dep	... 229899 μ s		21.749 MB/s
Running xbox	... 180639 μ s		27.680 MB/s
Running xbox_merged	... 142883 μ s		34.994 MB/s
Running zbb_default	... 203861 μ s		24.527 MB/s
Running zbb_xbox	... 144758 μ s		34.540 MB/s
--- 4 MB (4 MB, x2 repeats) ---			
Running default_loop	... 370999 μ s		21.563 MB/s
Running unrolled	... 331894 μ s		24.104 MB/s
Running reduced_dep	... 371496 μ s		21.535 MB/s

Figure 12: 对比测试各种优化方案效率

以默认循环实现 (default_loop) 为基准, 所有方案在每个数据规模下的运行时间 (elapsed_us)、吞吐率 (throughput_mbps) 和加速比 (speedup) 数据汇总如下。

Table 1: 各方案在不同数据规模下的运行时间 (单位: μs)

数据规模	运行时间 (elapsed_us)						
	default_loop	unrolled	reduced_dep	xbox	xbox_merged	zbb_default	zbb_xbox
1 KB	88,841	78,583	88,687	68,904	53,699	78,315	54,234
16 KB	142,796	126,458	142,336	110,931	86,501	126,095	87,517
64 KB	143,178	127,542	142,332	111,035	86,475	126,130	87,639
256 KB	228,803	203,336	228,606	178,773	140,014	202,658	142,426
1 MB	229,550	204,633	229,899	180,639	142,883	203,861	144,758
4 MB	370,999	331,894	371,496	293,106	236,653	329,914	239,037
16 MB	753,325	675,472	753,503	600,482	493,941	671,899	500,540

Table 2: 各方案在不同数据规模下的吞吐率 (单位: MB/s)

数据规模	吞吐率 (throughput_mbps)						
	default_loop	unrolled	reduced_dep	xbox	xbox_merged	zbb_default	zbb_xbox
1 KB	21.985	24.854	22.023	28.346	36.372	24.939	36.013
16 KB	21.884	24.712	21.955	28.171	36.127	24.783	35.707
64 KB	21.826	24.502	21.956	28.144	36.138	24.776	35.658
256 KB	21.853	24.590	21.872	27.968	35.711	24.672	35.106
1 MB	21.782	24.434	21.749	27.680	34.994	24.527	34.540
4 MB	21.563	24.104	21.535	27.294	33.805	24.249	33.468
16 MB	21.239	23.687	21.234	26.645	32.393	23.813	31.965

Table 3: 各方案在不同数据规模下的加速比 (基准: default_loop)

数据规模	加速比 (speedup, $\times$ vs baseline)					
	unrolled	reduced_dep	xbox	xbox_merged	zbb_default	zbb_xbox
1 KB	1.1305	1.0017	1.2893	1.6544	1.1344	1.6381
16 KB	1.1292	1.0032	1.2873	1.6508	1.1324	1.6316
64 KB	1.1226	1.0059	1.2895	1.6557	1.1352	1.6337
256 KB	1.1252	1.0009	1.2799	1.6341	1.1290	1.6065
1 MB	1.1218	0.9985	1.2708	1.6066	1.1260	1.5858
4 MB	1.1178	0.9987	1.2658	1.5677	1.1245	1.5521
16 MB	1.1153	0.9998	1.2545	1.5251	1.1212	1.5050

6 实验结果分析

6.1 整体性能对比

根据表3中加速比计算各方案在所有数据规模下的几何平均加速比, 结果如下:

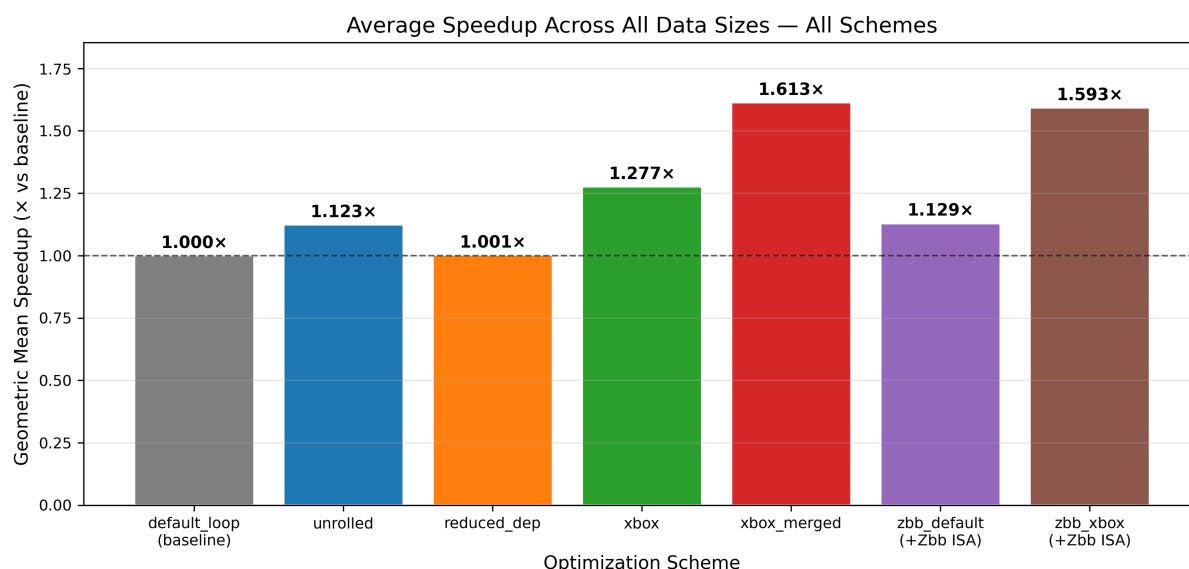


Figure 13: 各优化方案在所有数据规模下的几何平均加速比

从图13可以看出,7种方案按照性能提升从低到高排序为:

`reduced_dep` ($\approx 1.00\times$) < `unrolled` ($\approx 1.12\times$) < `zbb_default` ($\approx 1.13\times$) < `xbox` ($\approx 1.28\times$) < `zbb_xbox`

其中,`xbox_merged` 方案取得了最优的加速效果(几何平均加速比约 $1.61\times$),而 `reduced_dep` 方案几乎没有性能提升。

6.2 查表优化的压倒性优势

实验数据表明,基于预计算查表的优化方案(`xbox` 与 `xbox_merged`)是提升 SM4 加密性能最有效的手段。

`xbox` 方案的核心思想是将每轮迭代中需计算的 $T = L \circ S$ 函数预计算为一张 256 项的查找表。原始实现中,每轮需执行 1 次 S 盒查表、4 次循环左移和 4 次异或,共 9 条算术指令;而 `xbox` 方案将 T 函数的 32 轮中每轮操作合并为 4 次查表和 4 次异或,大幅缩减了指令数量。实验结果显示该方案取得了约 $1.28\times$ 的加速比。

`xbox_merged` 方案在此基础上进一步优化:将 4 张分别按移位偏移 (0, 2, 10, 18, 24) 索引的 256 项预计算表合并,每张表实际占用 1 KB。这种方法将每轮的 4 次查表 + 4 次异或进一步压缩为 4 次查表 + 3 次异或(合并了部分移位),同时减少了对四个独立 X_i 变量的存储需求,优化了寄存器分配。实验结果显示该方案取得了所有方法中最高的加速比(约 $1.61\times$),说明 SM4 加密算法的主要性能瓶颈在于每轮 T 函数中的大量移位与循环移位运算,而通过空间换时间的预计算查表策略能够从根本上消除这一瓶颈。

6.3 微架构优化的有限效果

6.3.1 循环展开 (unrolled)

unrolled 方案通过将 SM4 的 32 轮迭代在编译时展开为 32 段顺序代码, 消除了循环分支预测开销和循环计数器的自增/比较指令。实验结果显示加速比约为 1.12 $\times$, 效果有限。分析原因在于:

- SM4 的内层循环体本身比较简单, 循环开销占比低, 展开带来的绝对收益有限;
- RISC-V 8 核处理器拥有较大的 L1 指令缓存 (每核心 32 KiB), 展开后代码体积增大但未超出缓存容量, 因此无指令缓存缺失惩罚, 但也无法在缓存层面进一步获益。

6.3.2 降低指令依赖 (reduced_dep)

reduced_dep 方案尝试通过重排异或操作来打破原始实现中 X_i 变量之间的依赖链, 期望更充分利用处理器的指令级并行能力。然而实验结果显示加速比接近 1.00 $\times$ (甚至在某些大数据规模下略低于基准), 几乎完全没有效果。对此的解释是:

- Ky(R) X1 处理器为顺序发射的 RISC-V 核心, 指令级并行能力有限, 打破依赖链不能有效提升每周指令发射数;
- SM4 轮函数中的关键路径是 S 盒查表和 T 函数的线性变换, 异或操作的依赖链并非性能瓶颈, 打破这部分依赖无助于缩短关键路径;
- 变量重排可能干扰了编译器的寄存器分配策略, 反而在部分场景下产生了轻微的负优化效果 (大块数据下 $< 1.00\times$)。

6.4 ISA 扩展的独立与叠加效应

RISC-V Zbb 位操作扩展提供了两条对 SM4 优化有益的指令:

- rev8: 单指令完成 32 位字的字节序反转, 替代原始 sm4LoadBlock 中的 4 次 lbu+slli + or 序列;
- rori: 单指令完成一次循环右移, 替代原始 L 变换中每个循环移位所需的 2 条移位 +1 条或指令。

实验数据显示:

- 独立应用 Zbb 扩展: zbb_default 方案相对于 default_loop 取得约 1.13 $\times$ 的加速比, 与 unrolled 相当, 说明单纯替换几条位操作指令对整体性能提升有限;
- 叠加应用 Zbb 扩展: zbb_xbox 方案 ($\approx 1.59\times$) 虽然较 zbb_default 提升显著, 但仍低于 xbox_merged ($\approx 1.61\times$)。这是因为 Zbb 优化的 rev8、rori 指令主要加速字节序转换和循环移位操作, 而 xbox_merged 已经将这些操作转移到编译时完成, 运行时仅剩查表和异或, 因此 Zbb 扩展在查表优化方案中的边际收益很小。

6.5 数据规模对加速比的影响

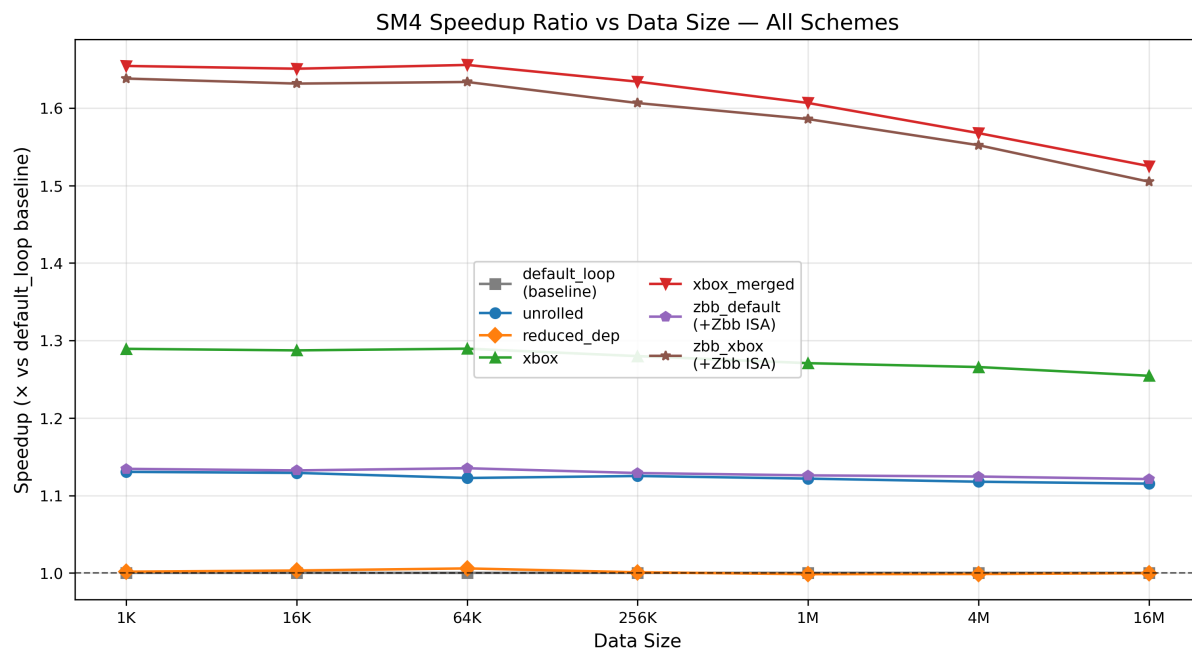


Figure 14: 各方案加速比随数据规模变化的趋势

从图14和表2可以观察到两个趋势:

1. 吞吐率随数据规模增大而轻微下降: 以基准方案为例, 吞吐率从 1 KB 时的 21.985 MB/s 缓慢下降到 16 MB 时的 21.239 MB/s(降幅约 3.4%)。这一趋势在所有方案中均存在, 原因在于数据规模增大后 Cache 命中率下降,L2 缓存容量 (每 4 核心共享 512 KiB) 相对有限, 大块数据处理时访存瓶颈逐渐显现。
2. 加速比随数据规模增大而轻微递减: 所有方案的加速比在大数据规模下均出现小幅衰减。例如 `xbox_merged` 从 1 KB 时的 $1.654\times$ 下降到 16 MB 时的 $1.525\times$ 。这说明查表优化方案的优势部分依赖于 L1 数据缓存的高命中率; 当数据量超过 L1/L2 缓存容量时, 表查找的频率不变但数据的加载存储开销增大, 稀释了查表优化的相对优势。

6.6 小结与建议

综合以上分析, 可以得出以下结论:

1. 预计算查表 (空间换时间) 是优化 SM4 加密算法在 RISC-V 平台上最有效的手段,`xbox_merged` 方案可取得约 $1.61\times$ 的加速;
2. 单纯的指令级优化 (循环展开、降低依赖) 和 ISA 扩展 (Zbb) 对性能提升有限 (均不超过 $1.13\times$),SM4 的性能瓶颈在于算法的计算结构本身而非指令编码效率;
3. 不同优化方案的叠加并非总能产生 $1 + 1 > 2$ 的效果, 如 Zbb 扩展在查表方案中的边际收益极小, 实际工程中应优先选择效果最显著的单一优化手段;

4. 在资源受限的嵌入式 RISC-V 平台上,4 KB 的预计算表空间代价完全可以接受,建议在工程实践中采用 `xbox_merged` 方案。